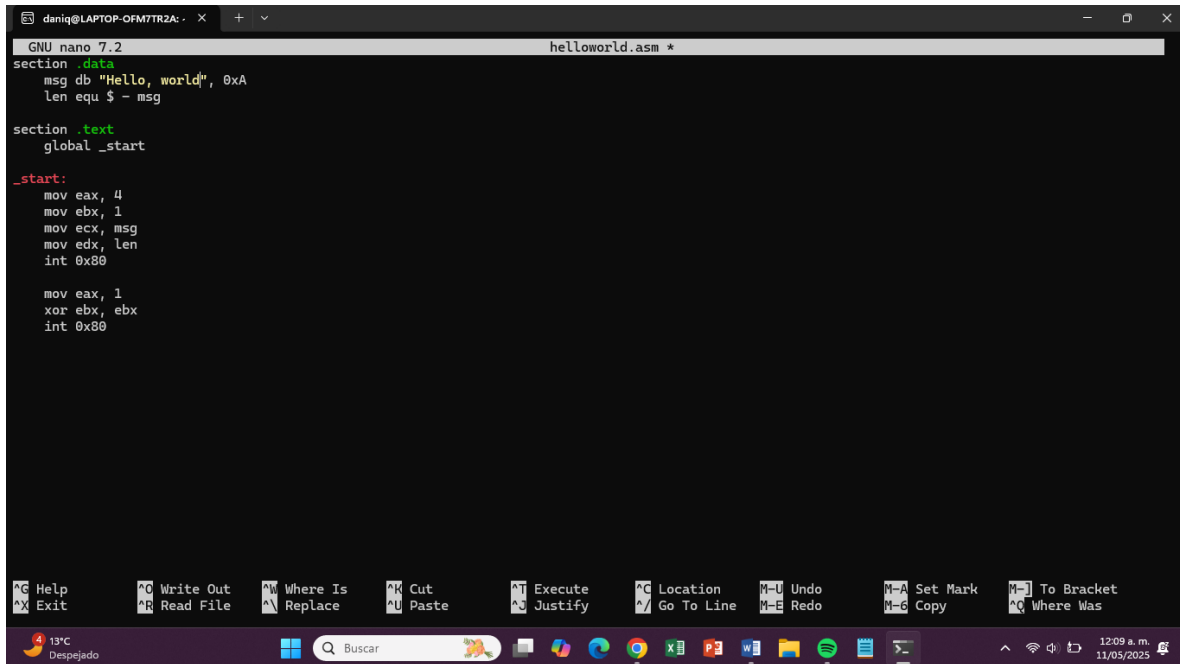


Código 1: Hello World

Código:



The screenshot shows a Windows desktop environment with a terminal window open. The terminal window has a title bar that reads 'daniq@LAPTOP-OFM7TR2A:'. Inside the terminal, the GNU nano 7.2 text editor is running, editing a file named 'helloworld.asm'. The code in the editor is as follows:

```
section .data
    msg db "Hello, world!", 0xA
    len equ $ - msg

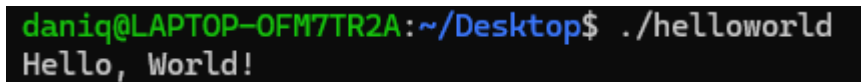
section .text
    global _start

_start:
    mov eax, 4
    mov ebx, 1
    mov ecx, msg
    mov edx, len
    int 0x80

    mov eax, 1
    xor ebx, ebx
    int 0x80
```

Below the code, the nano editor's menu bar is visible, showing options like 'Help', 'Exit', 'Write Out', 'Read File', 'Where Is', 'Replace', 'Cut', 'Paste', 'Execute', 'Justify', 'Location', 'Go To Line', 'Undo', 'Redo', 'Set Mark', 'Copy', 'To Bracket', and 'Where Was'. At the bottom of the terminal window, the Windows taskbar is visible, showing the system clock as 12:09 a.m. on 11/05/2025.

Ejecución:

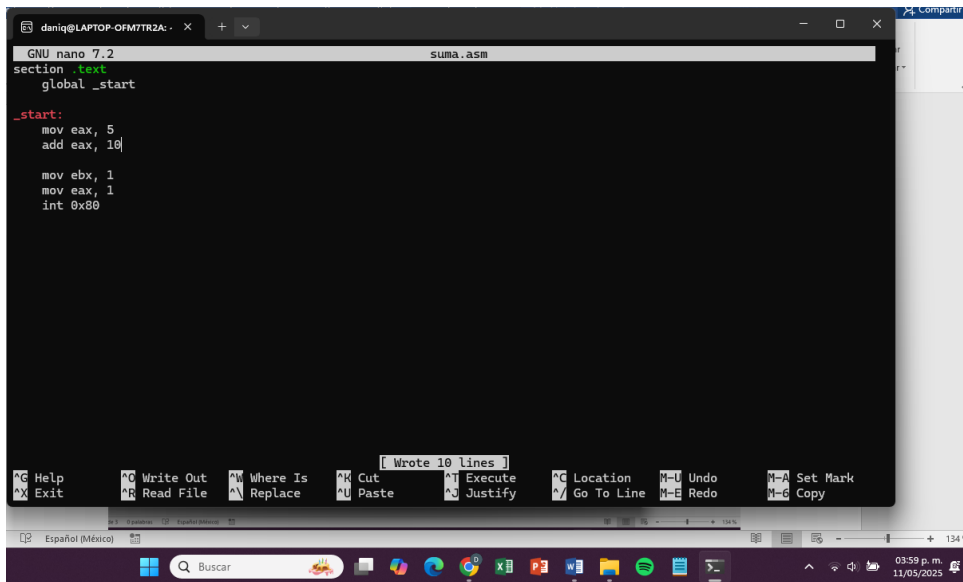


The screenshot shows a terminal window with the following text:

```
daniq@LAPTOP-OFM7TR2A:~/Desktop$ ./helloworld
Hello, World!
```

Código 2: Sumar dos números

Código:



The screenshot shows a terminal window with the GNU nano 7.2 editor open. The file being edited is suma.asm. The code in the editor is as follows:

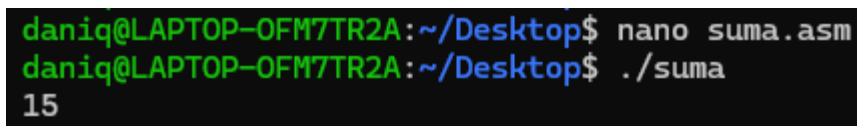
```
section .text
global _start

_start:
    mov eax, 5
    add eax, 10

    mov ebx, 1
    mov eax, 1
    int 0x80
```

The terminal window also shows a status bar at the bottom with various keyboard shortcuts and a message that says "Wrote 10 lines". The Windows taskbar is visible at the bottom of the screen.

Ejecución:



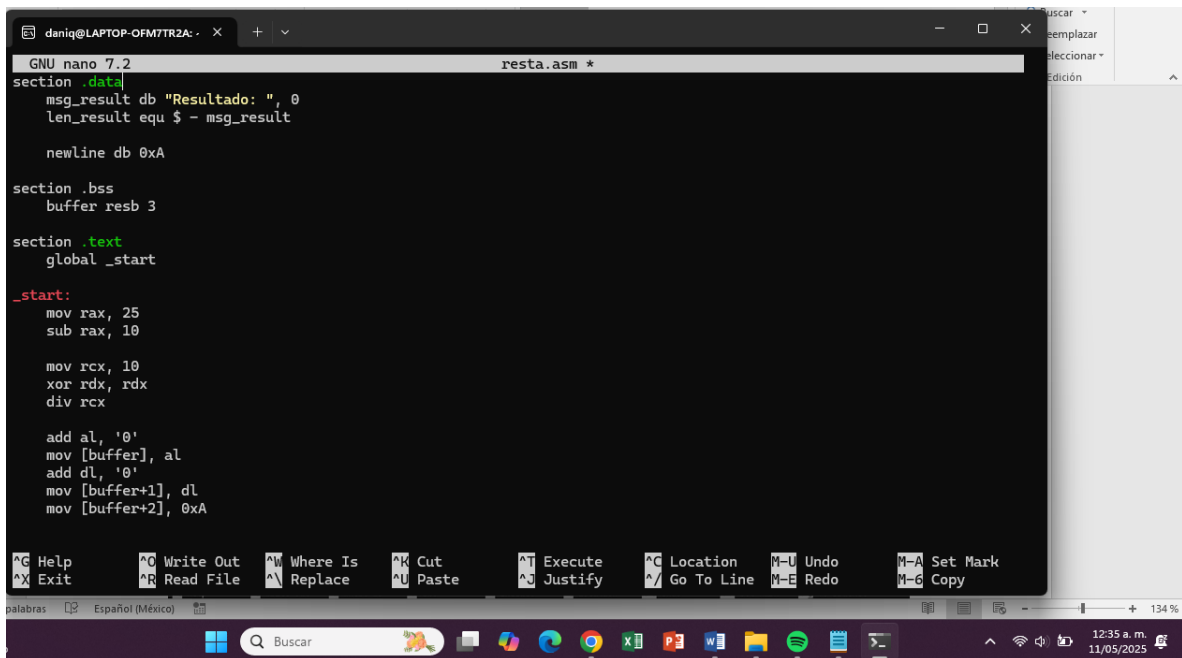
The screenshot shows a terminal window with the following commands and output:

```
daniq@LAPTOP-OFM7TR2A:~/Desktop$ nano suma.asm
daniq@LAPTOP-OFM7TR2A:~/Desktop$ ./suma
15
```

The output of the program is the number 15, which is the sum of 5 and 10.

Código 3: Restar dos números

Código:



```
GNU nano 7.2          resta.asm *
section .data
msg_result db "Resultado: ", 0
len_result equ $ - msg_result

newline db 0xA

section .bss
buffer resb 3

section .text
global _start

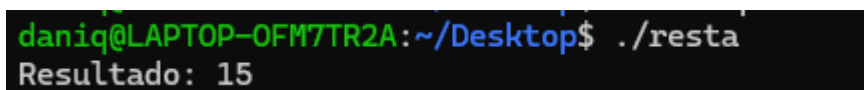
_start:
mov rax, 25
sub rax, 10

mov rcx, 10
xor rdx, rdx
div rcx

add al, '0'
mov [buffer], al
add dl, '0'
mov [buffer+1], dl
mov [buffer+2], 0xA

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^N Replace   ^U Paste     ^J Justify
^C Location  ^_ Go To Line ^M Undo     ^- Redo      ^- Set Mark
^M Copy
```

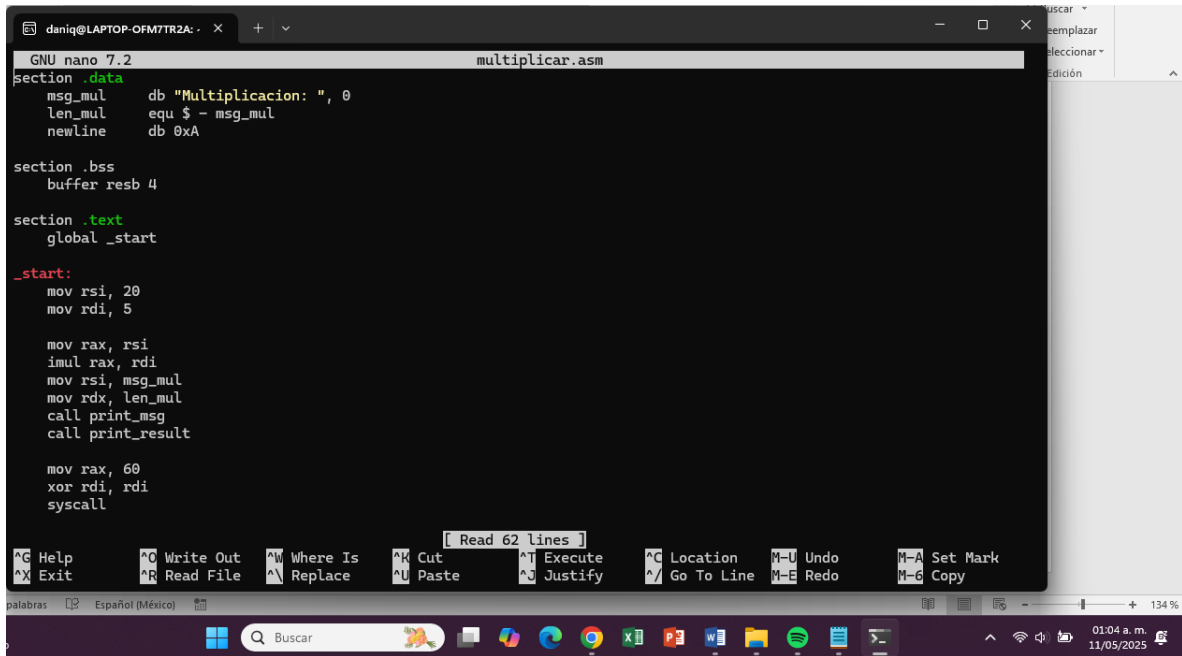
Ejecución:



```
daniq@LAPTOP-OFM7TR2A: ~/Desktop$ ./resta
Resultado: 15
```

Código 4: Multiplicación

Código:



```
GNU nano 7.2      multiplicar.asm
section .data
msg_mul    db "Multiplicacion: ", 0
len_mul    equ $ - msg_mul
newline    db 0xA

section .bss
buffer resb 4

section .text
global _start

_start:
mov rsi, 20
mov rdi, 5

mov rax, rsi
imul rax, rdi
mov rsi, msg_mul
mov rdx, len_mul
call print_msg
call print_result

mov rax, 60
xor rdi, rdi
syscall

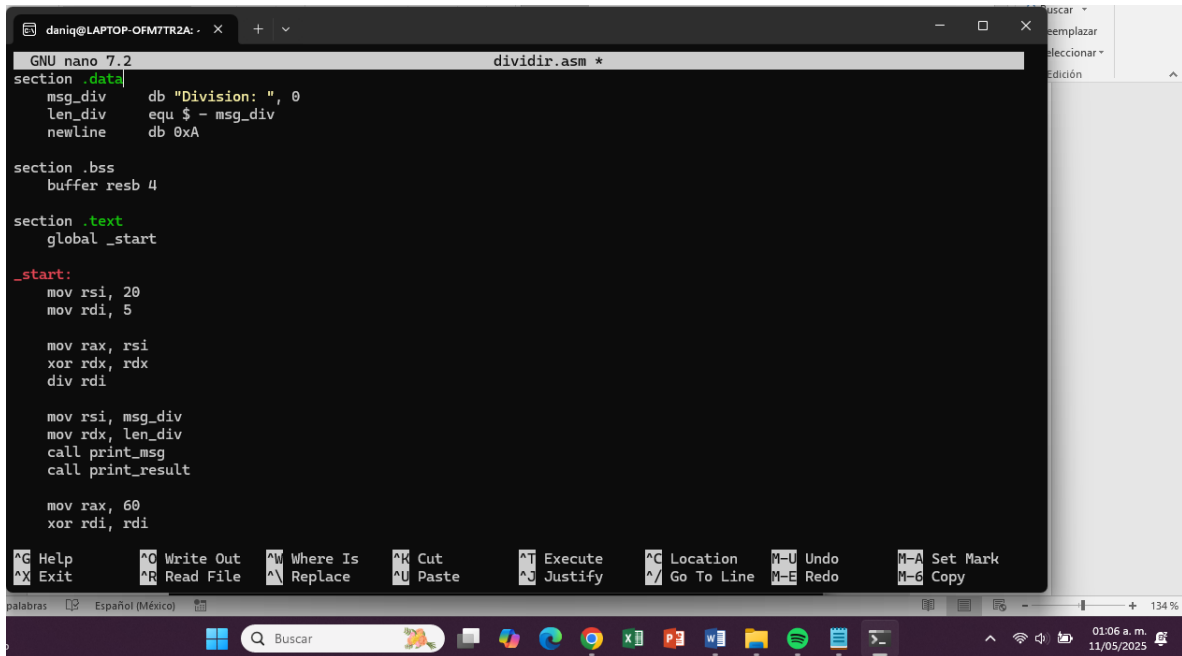
[ Read 62 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^J Execute
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^_ Justify
^C Location   ^M Undo       ^- Set Mark
^_ Go To Line ^E Redo       ^G Copy
```

Ejecución:

```
daniq@LAPTOP-OFM7TR2A:~/Desktop$ ./multiplicar
Multiplicacion: 100
```

Código 5: División

Código:



```
GNU nano 7.2 dividir.asm *
section .data
    msg_div    db "Division: ", 0
    len_div    equ $ - msg_div
    newline    db 0xA

section .bss
    buffer resb 4

section .text
    global _start

_start:
    mov rsi, 20
    mov rdi, 5

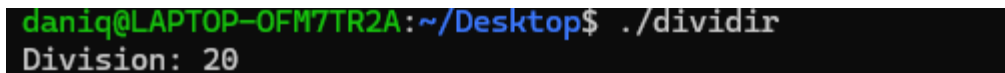
    mov rax, rsi
    xor rdx, rdx
    div rdi

    mov rsi, msg_div
    mov rdx, len_div
    call print_msg
    call print_result

    mov rax, 60
    xor rdi, rdi

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^N Replace   ^U Paste     ^C Justify
^_ Location  ^_ Go To Line ^- Undo     ^- Redo      ^- Set Mark
^_ Copy
```

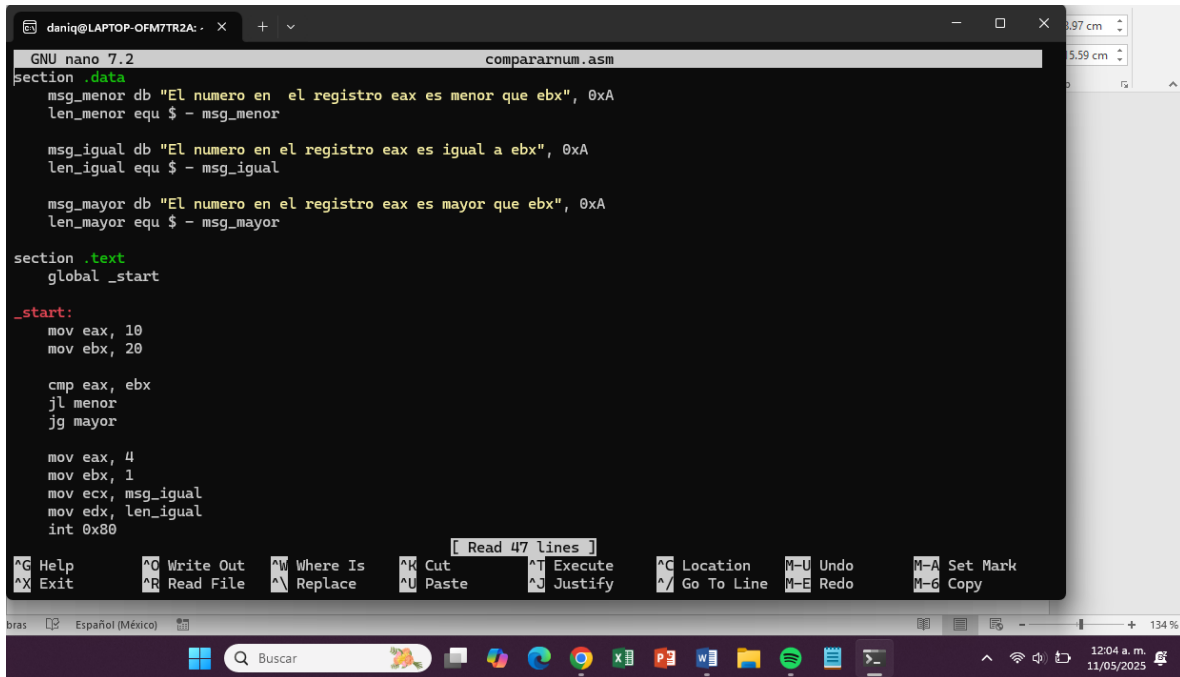
Ejecución:



```
daniq@LAPTOP-OFM7TR2A:~/Desktop$ ./dividir
Division: 20
```

Código 6: Comparar igualdad, mayor que o menor que

Código:



```
GNU nano 7.2 compararnum.asm
section .data
    msg_menor db "El numero en el registro eax es menor que ebx", 0xA
    len_menor equ $ - msg_menor

    msg_igual db "El numero en el registro eax es igual a ebx", 0xA
    len_igual equ $ - msg_igual

    msg_mayor db "El numero en el registro eax es mayor que ebx", 0xA
    len_mayor equ $ - msg_mayor

section .text
    global _start

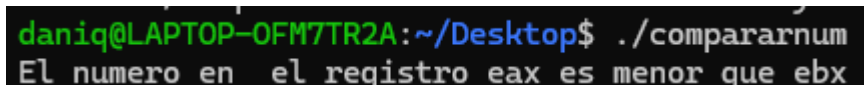
_start:
    mov eax, 10
    mov ebx, 20

    cmp eax, ebx
    jl menor
    jg mayor

    mov eax, 4
    mov ebx, 1
    mov ecx, msg_igual
    mov edx, len_igual
    int 0x80

[ Read 47 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  ^M-U Undo    ^M-A Set Mark
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/_ Go To Line ^M-E Redo    ^M-G Copy
```

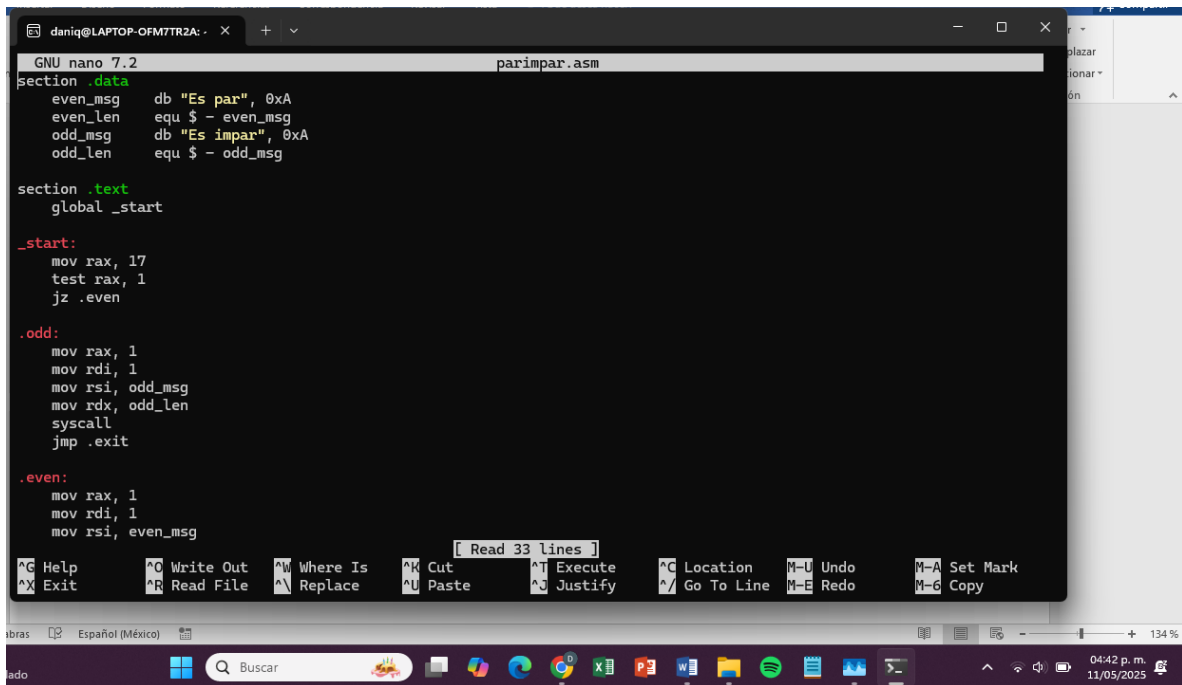
Ejecución:



```
daniq@LAPTOP-OFM7TR2A:~/Desktop$ ./compararnum
El numero en el registro eax es menor que ebx
```

Código 7: Comprobación de numero par o impar

Código:



```
GNU nano 7.2 parimpar.asm
section .data
    even_msg db "Es par", 0xA
    even_len equ $ - even_msg
    odd_msg db "Es impar", 0xA
    odd_len equ $ - odd_msg

section .text
    global _start

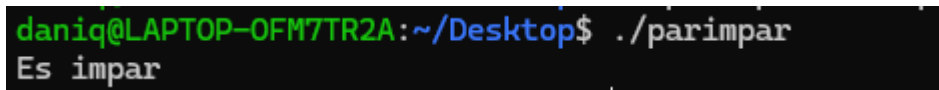
_start:
    mov rax, 17
    test rax, 1
    jz .even

.odd:
    mov rax, 1
    mov rdi, 1
    mov rsi, odd_msg
    mov rdx, odd_len
    syscall
    jmp .exit

.even:
    mov rax, 1
    mov rdi, 1
    mov rsi, even_msg
    syscall
    jmp .exit

[ Read 33 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo    M-A Set Mark
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_/ Go To Line M-E Redo    M-6 Copy
```

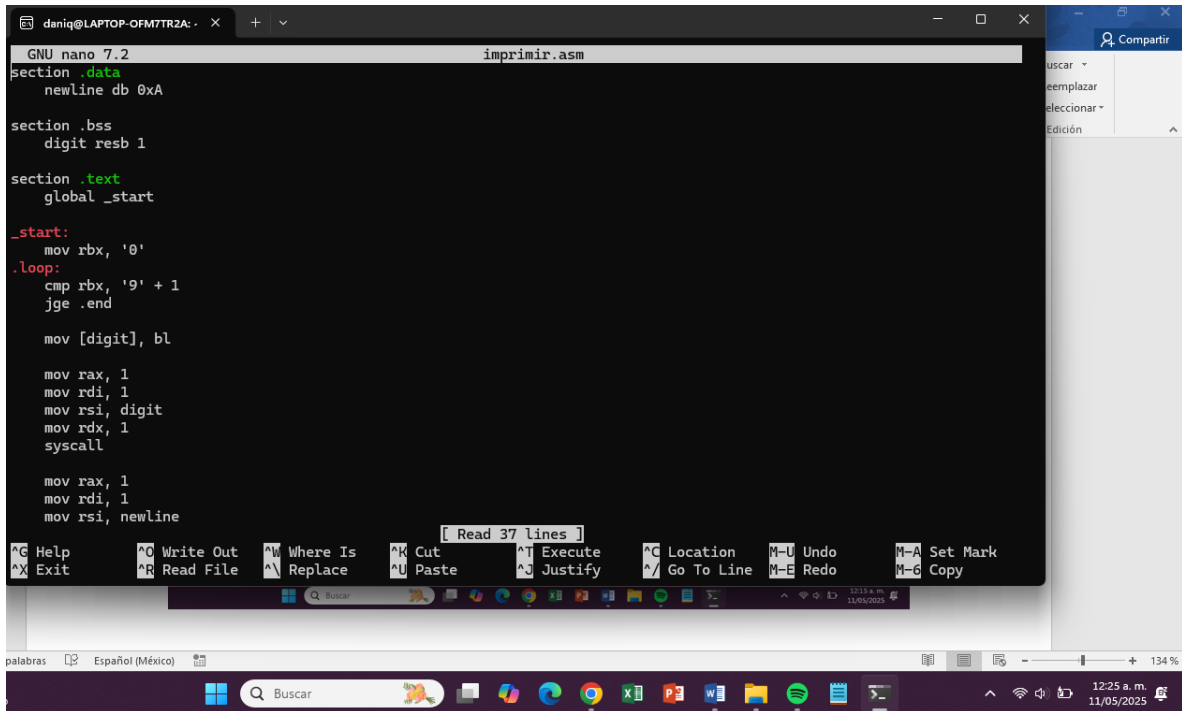
Ejecución:



```
daniq@LAPTOP-OFM7TR2A:~/Desktop$ ./parimpar
Es impar
```

Código 8: Imprimir con loop

Código:



```
GNU nano 7.2 imprimir.asm
section .data
    newline db 0xA

section .bss
    digit resb 1

section .text
    global _start

_start:
    mov rbx, '0'
.loop:
    cmp rbx, '9' + 1
    jge .end

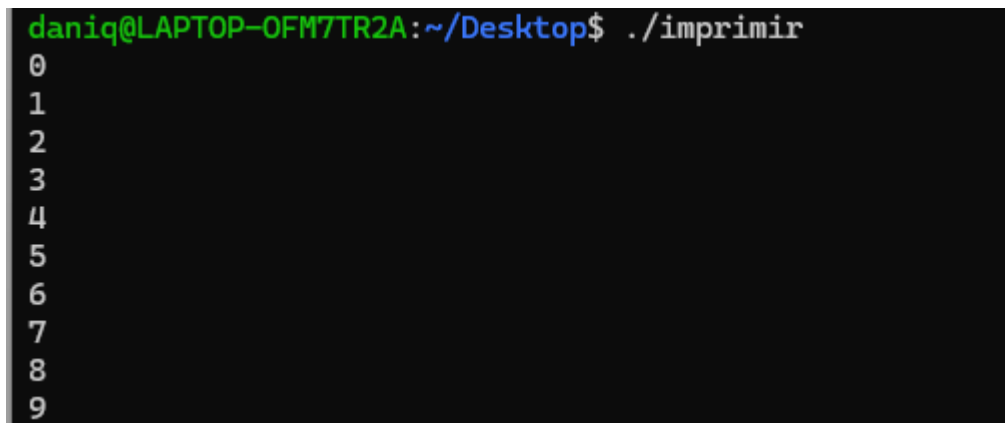
    mov [digit], bl

    mov rax, 1
    mov rdi, 1
    mov rsi, digit
    mov rdx, 1
    syscall

    mov rax, 1
    mov rdi, 1
    mov rsi, newline

    [ Read 37 lines ]
    ^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo     M-A Set Mark
    ^X Exit      ^R Read File  ^E Replace    ^U Paste      ^J Justify    ^/_ Go To Line M-E Redo     M-G Copy
    palabras    Español (México)
    Buscar
```

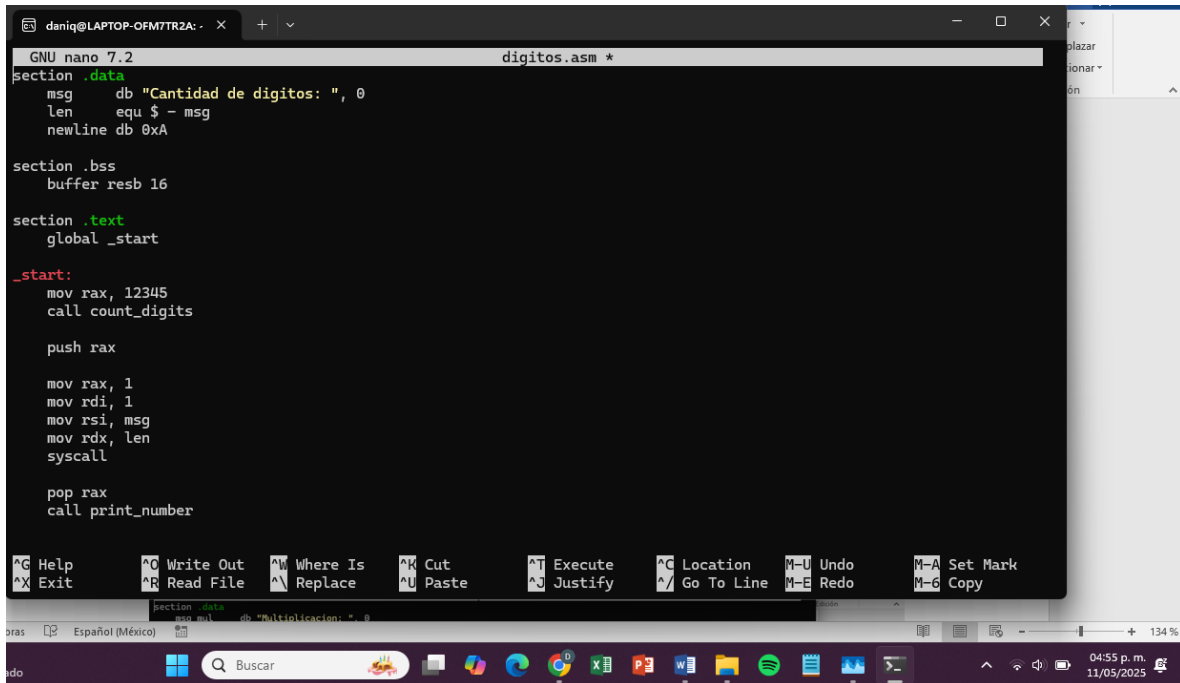
Ejecución:



```
daniq@LAPTOP-OFM7TR2A:~/Desktop$ ./imprimir
0
1
2
3
4
5
6
7
8
9
```


Código 9: Contador de dígitos

Código:



```
GNU nano 7.2 digitos.asm *
section .data
msg db "Cantidad de digitos: ", 0
len equ $ - msg
newline db 0xA

section .bss
buffer resb 16

section .text
global _start

_start:
mov rax, 12345
call count_digits

push rax

mov rax, 1
mov rdi, 1
mov rsi, msg
mov rdx, len
syscall

pop rax
call print_number

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
^_          ^C Location   ^M-U Undo     ^-A Set Mark
           ^/ Go To Line ^M-E Redo     ^M-6 Copy
```

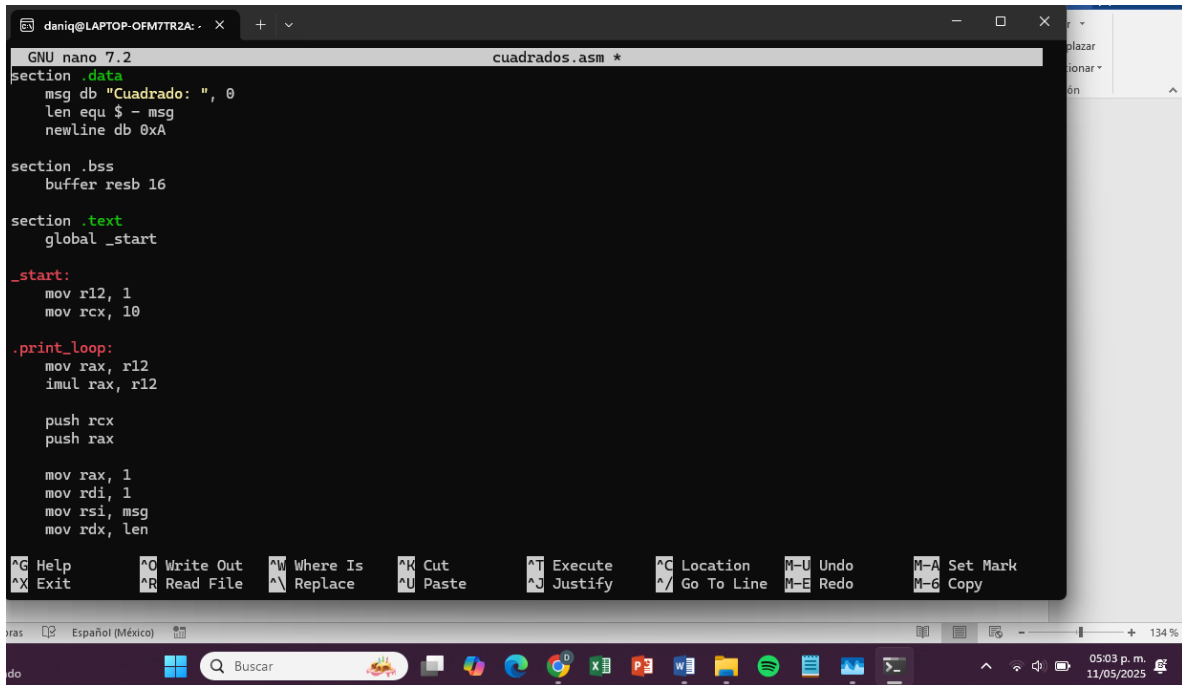
Ejecución:



```
daniq@LAPTOP-OFM7TR2A:~/Desktop$ ./digitos
Cantidad de digitos: 5
```

Código 10: Obtención del cuadrado de un numero

Código:



```
GNU nano 7.2 cuadrados.asm *
section .data
    msg db "Cuadrado: ", 0
    len equ $ - msg
    newline db 0xA

section .bss
    buffer resb 16

section .text
    global _start

_start:
    mov r12, 1
    mov rcx, 10

.print_loop:
    mov rax, r12
    imul rax, r12

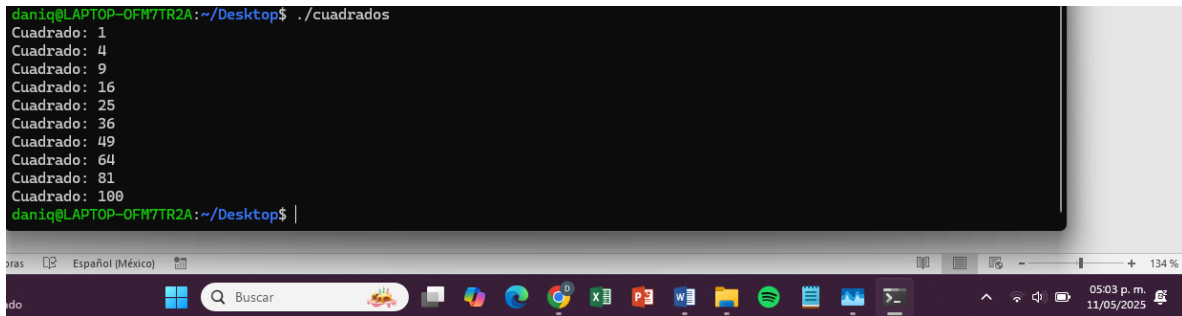
    push rcx
    push rax

    mov rax, 1
    mov rdi, 1
    mov rsi, msg
    mov rdx, len

    Help      Write Out  Where Is  Cut      Execute  Location  M-U  Undo  M-A  Set Mark
    Exit      Read File  Replace  Paste    Justify  Go To Line M-E  Redo  M-G  Copy

    rax      Español (México)
    do       Buscar
    05:03 p. m. 11/05/2025
```

Ejecución:



```
daniq@LAPTOP-OFM7TR2A:~/Desktop$ ./cuadrados
Cuadrado: 1
Cuadrado: 4
Cuadrado: 9
Cuadrado: 16
Cuadrado: 25
Cuadrado: 36
Cuadrado: 49
Cuadrado: 64
Cuadrado: 81
Cuadrado: 100
daniq@LAPTOP-OFM7TR2A:~/Desktop$
```